



ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
Khoa Công nghệ Thông tin

NHẬP MÔN HỌC MÁY

Báo cáo chi tiết của Bài báo

“ReLation Graph Transformer (RelGT)”

Nhóm sinh viên:

Nguyễn Đình Hà Dương - 23122002

Bùi Duy Bảo - 23122021

Nguyễn Việt Hùng 23122032

Giảng viên:

PGS.TS. Lê Hoàng Thái

ThS. Huỳnh Lâm Hải Đăng

GVHD Nguyễn Thanh Tình

Ngày nộp:

Thứ Tư, 2 tháng 5 năm 2026

Mục lục

1	Giới thiệu	2
2	Bối cảnh: Relational Deep Learning	2
2.1	Dữ liệu quan hệ và giới hạn của cách tiếp cận cũ	2
2.2	Vì sao GNN truyền thống chưa đủ	3
2.3	Relational Entity Graph	3
3	RelGT (Relation Graph Transformer)	4
3.1	Tổng quan kiến trúc	4
3.2	Lấy mẫu lân cận nhận biết thời gian	4
3.3	Token hóa 5 thành phần	4
3.4	Encoder đặc trưng bằng	5
3.5	Time encoder	5
3.6	GNN positional encoding	5
3.7	Local Transformer	6
3.8	Global centroid module	6
3.9	Kết hợp local và global	6
3.10	Đánh giá trên RelBench	6
4	RelGT++: Phiên bản Cải tiến	8
4.1	Mục tiêu thiết kế	8
4.2	CrossModalGatedFusion	8
4.3	SwiGLU Feed-Forward Network	8
4.4	Stochastic Depth	8
4.5	Multi-View Readout	9
4.6	GumbelSoftCodebook	9
4.7	CrossAttentionBridge	9
4.8	Kỹ thuật triển khai dữ liệu	9
5	Cấu hình tham khảo so sánh giữa RelGT và RelGT++	10
6	RelGT-X: Hướng Mở rộng Nghiên cứu	11
6.1	Động lực	11
6.2	Schema-Guided Sparse Attention	11
6.3	Causal Temporal Disentanglement	11
6.4	Hierarchical Codebook với Schema Priors	11
6.5	Relational Positional Encoding	12
6.6	Meta-Learning Tokenization	12
7	Thảo luận	12
7.1	Ý nghĩa khoa học	12
7.2	Giới hạn	13
7.3	Kết luận	13

1 Giới thiệu

Ý tưởng của bài báo

RELGT là kiến trúc Graph Transformer được thiết kế cho dữ liệu quan hệ đa bảng, được giới thiệu trong paper gốc *Relational Graph Transformer* của Vijay Prakash Dwivedi, Sri Jaladi, Yangyi Shen, Federico López, Charilaos I. Kanatsoulis, Rishi Puri, Matthias Fey và Jure Leskovec (arXiv:2505.10960v1, 16/05/2025) [1].

Paper này đặt vấn đề rằng dữ liệu doanh nghiệp thực tế thường không tồn tại như một bảng đơn lẻ, mà là một hệ nhiều bảng, nhiều kiểu thực thể, nhiều quan hệ khóa chính–khóa ngoại và nhiều mốc thời gian. Vì vậy, mô hình cần học đồng thời đặc trưng bảng, cấu trúc quan hệ, ngữ nghĩa schema và động lực thời gian.

Khác với GNN truyền thống, RELGT biến mỗi node trong đồ thị quan hệ thành một token giàu ngữ cảnh gồm đặc trưng bảng, loại node, khoảng cách hop, thời gian tương đối và mã hóa vị trí cục bộ. Sau đó mô hình dùng attention cục bộ để học tương tác trong đồ thị con và attention toàn cục đến một tập centroid để thu nhận ngữ cảnh xa.

Báo cáo này không chỉ tóm tắt paper gốc, mà còn mở rộng phân tích theo ba thể hệ kiến trúc:

- **RelGT gốc:** đóng góp nền tảng về token hóa 5 thành phần, lấy mẫu nhận biết thời gian, local transformer và global centroid.
- **RelGT++:** phiên bản triển khai/cải tiến, tăng tính ổn định và biểu đạt bằng CrossModalGatedFusion, SwiGLU, DropPath, Multi-View Readout, GumbelSoftCodebook và CrossAttention-Bridge.
- **RelGT-X:** hướng nghiên cứu kế tiếp, đưa schema prior, temporal causal disentanglement, hierarchical codebook, relational positional encoding, meta-learning tokenization và contrastive schema alignment vào cùng một khung.

Nhìn tổng thể, RELGT giải quyết một nút thắt rất thực tế: dữ liệu doanh nghiệp không nằm trong một ma trận duy nhất mà trong nhiều bảng, nhiều kiểu thực thể, nhiều quan hệ khóa chính–khóa ngoại và nhiều mốc thời gian. Bài toán không chỉ là học đặc trưng tabular, cũng không chỉ là học graph; nó là học trên một *hệ quan hệ có cấu trúc, có thời gian và có ngữ nghĩa schema*.

2 Bối cảnh: Relational Deep Learning

2.1 Dữ liệu quan hệ và giới hạn của cách tiếp cận cũ

Trong các hệ thống thương mại điện tử, tài chính, y tế, mạng xã hội hoặc vận hành doanh nghiệp, dữ liệu thường được lưu trong cơ sở dữ liệu quan hệ: bảng người dùng, bảng sản phẩm, bảng đơn hàng, bảng đánh giá, bảng sự kiện, bảng thanh toán, v.v. Các bảng này được nối bằng khóa chính và khóa ngoại.

Cách làm truyền thống thường gồm ba bước: chọn bảng mục tiêu, join nhiều bảng liên quan, sinh đặc trưng thủ công, rồi huấn luyện một mô hình tabular như LightGBM hoặc mạng neural cho bảng phẳng. Cách làm đó có ba hạn chế lớn:

1. **Tổn công thiết kế đặc trưng:** cần chuyên gia hiểu nghiệp vụ, schema và cửa sổ thời gian.
2. **Mất cấu trúc quan hệ:** sau khi join/aggregate, nhiều đường đi quan hệ trong schema bị nén thành thống kê đơn giản.
3. **Ễ rò rỉ thời gian:** nếu aggregation dùng dữ liệu sau thời điểm dự báo, mô hình nhìn thấy tương lai.

2.2 Vì sao GNN truyền thông chưa đủ

GNN học bằng message passing: sau L lớp, một node nhận thông tin từ vùng L -hop. Trên dữ liệu quan hệ, điều này có thể thiếu linh hoạt:

- Phụ thuộc xa cần đi qua nhiều bảng trung gian nên dễ bị *over-squashing*.
- Hai thực thể cùng kiểu có thể liên quan qua schema nhưng không có cạnh trực tiếp trong đồ thị con nhỏ.
- Heterogeneity và temporal dynamics thường bị nhét vào cùng một vector, làm mất ranh giới ngữ nghĩa giữa nội dung, kiểu node, thời gian và vị trí.

Trực giác

GNN truyền thông giải lan truyền tín hiệu theo cạnh. RELGT đặt câu hỏi khác: nếu đã có một đồ thị con quanh seed node, ta có thể xem các node trong đó như một tập token và để Transformer học mọi tương tác quan trọng giữa chúng không?

2.3 Relational Entity Graph

Định nghĩa 2.1 (Relational Entity Graph). Một REG được định nghĩa là:

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}, \varphi, \psi, \tau),$$

trong đó $\mathcal{V}$ là tập node/thực thể, $\mathcal{E}$ là tập cạnh quan hệ, $\varphi : \mathcal{V} \rightarrow \mathcal{T}$ ánh xạ node về loại bảng, $\psi : \mathcal{E} \rightarrow \mathcal{R}$ ánh xạ cạnh về loại quan hệ, và $\tau : \mathcal{V} \rightarrow \mathbb{R}$ là thời điểm của node hoặc sự kiện.

REG có ba đặc điểm quan trọng:

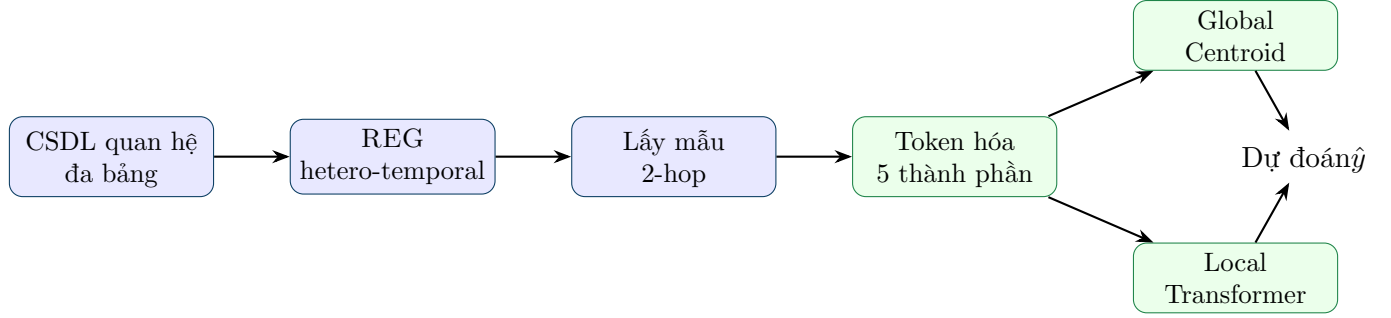
- **Schema-defined**: edge không tùy ý mà do schema cơ sở dữ liệu quy định.
- **Temporal**: dự báo phải tôn trọng thứ tự thời gian, đặc biệt trong các tác vụ như churn, fraud, sales forecasting.
- **Heterogeneous**: mỗi bảng có miền đặc trưng, kiểu dữ liệu và ý nghĩa nghiệp vụ riêng.

3 RelGT (Relation Graph Transformer)

3.1 Tổng quan kiến trúc

RELGT gồm ba bước chính:

REG $\xrightarrow{\text{lấy mẫu nhận biết thời gian}}$ đồ thị con quanh seed $\xrightarrow{\text{token hóa 5 thành phần}}$ local + global Transformer $\rightarrow \hat{y}$.



Hình 1: Pipeline khái quát của RELGT.

3.2 Lấy mẫu lân cận nhận biết thời gian

Với seed node v_i , RELGT lấy $K - 1$ node lân cận trong phạm vi 2-hop, nhưng chỉ giữ những node không xảy ra sau seed:

$$\mathcal{N}(v_i) = \{v_j : d(v_i, v_j) \leq 2, \tau(v_j) \leq \tau(v_i)\}.$$

Nếu số lân cận không đủ, mô hình dùng lấy mẫu có hoàn lại; nếu không có lân cận hợp lệ, mô hình dùng fallback token từ toàn đồ thị.

Quy tắc lấy mẫu

$$\mathcal{S}(v_i) = \begin{cases} \{v_i\} \cup \text{Sample}(\mathcal{N}(v_i), K - 1), & |\mathcal{N}(v_i)| \geq K - 1, \\ \{v_i\} \cup \text{SampleWithReplacement}(\mathcal{N}(v_i), K - 1), & 0 < |\mathcal{N}(v_i)| < K - 1, \\ \{v_i\} \cup \text{FallbackSample}(\mathcal{V}, K - 1), & |\mathcal{N}(v_i)| = 0. \end{cases}$$

Ràng buộc $\tau(v_j) \leq \tau(v_i)$ là yếu tố rất quan trọng. Nó biến quá trình lấy mẫu từ một thao tác graph thông thường thành một thao tác phù hợp cho dự báo theo thời gian.

3.3 Token hóa 5 thành phần

Mỗi token $t(v_j | v_i)$ trong ngữ cảnh seed v_i được biểu diễn bằng:

$$t(v_j | v_i) = (x_{v_j}, \varphi(v_j), p(v_i, v_j), \tau(v_j) - \tau(v_i), \text{GNN-PE}_{v_j}).$$

Bảng 1: Năm thành phần token trong RELGT.

#	Thành phần	Vai trò
1	Node feature x_{v_j}	Nội dung bảng: số, phân loại, timestamp, văn bản/embedding.
2	Node type $\varphi(v_j)$	Cho biết token đến từ bảng nào, giúp mô hình phân biệt ngữ nghĩa thực thể.
3	Hop distance $p(v_i, v_j)$	Mô tả khoảng cách cấu trúc so với seed: 1-hop, 2-hop hoặc fallback.
4	Relative time $\Delta\tau$	Giữ thông tin thứ tự và khoảng cách thời gian giữa token và seed.
5	Subgraph PE	Mã hóa vị trí cục bộ bằng GNN nhẹ trên đồ thị con.

Các thành phần được mã hóa riêng:

$$h_{\text{feat}}(v_j) = \text{MultiModalEncoder}(x_{v_j}), \quad (1)$$

$$h_{\text{type}}(v_j) = W_{\text{type}}[\varphi(v_j)], \quad (2)$$

$$h_{\text{hop}}(v_i, v_j) = W_{\text{hop}}[p(v_i, v_j)], \quad (3)$$

$$h_{\text{time}}(v_i, v_j) = W_{\text{time}} \text{PE}(\tau(v_j) - \tau(v_i)), \quad (4)$$

$$h_{\text{pe}}(v_j) = \text{GNN}(A_{\text{local}}, Z_{\text{random}})_j. \quad (5)$$

Sau đó:

$$h_{\text{token}}(v_j) = O[h_{\text{feat}} \parallel h_{\text{type}} \parallel h_{\text{hop}} \parallel h_{\text{time}} \parallel h_{\text{pe}}],$$

với O là ma trận trộn có thể học.

3.4 Encoder đặc trưng bảng

Đặc trưng bảng trong dữ liệu quan hệ thường đa phương thức: số, phân loại, multi-categorical, timestamp, text embedding. RELGT dùng encoder theo kiểu dữ liệu, rồi đưa về cùng chiều d :

$$h_{\text{feat}}^{(t)} = \text{ResNet}^{(t)}(\text{Enc}_{\text{num}}, \text{Enc}_{\text{cat}}, \text{Enc}_{\text{text}}, \text{Enc}_{\text{time}}).$$

Điểm đáng chú ý là encoder được tách theo loại bảng t . Điều này phù hợp vì cùng một cột số trong bảng sản phẩm và bảng người dùng không nhất thiết có cùng phân phối hoặc cùng ý nghĩa.

3.5 Time encoder

RELGT dùng positional encoding kiểu Transformer cho thời gian tương đối:

$$\text{PE}(\Delta\tau)_{2k} = \sin\left(\frac{\Delta\tau}{10000^{2k/d}}\right), \quad \text{PE}(\Delta\tau)_{2k+1} = \cos\left(\frac{\Delta\tau}{10000^{2k/d}}\right).$$

Sau đó:

$$h_{\text{time}} = W_{\text{time}} \text{PE}(\Delta\tau) + b_{\text{time}}.$$

Với token không có thời gian hợp lệ, triển khai có thể dùng một mask vector có thể học để không ép giá trị thiếu thành một thời điểm giả.

3.6 GNN positional encoding

GNN-PE chạy một GNN nhẹ trên đồ thị con:

$$h_{\text{pe}}(v_j) = \text{GNN}(A_{\text{local}}, Z_{\text{random}})_j.$$

Z_{random} được lấy ngẫu nhiên để phá đối xứng giữa các vị trí cấu trúc. Về trực giác, nếu hai node có feature giống nhau nhưng nằm ở vị trí cấu trúc khác nhau trong đồ thị con, GNN-PE tạo thêm tín hiệu giúp Transformer phân biệt.

Nhận xét 3.1. *GNN-PE là cầu nối giữa GNN và Transformer. Nó không dùng GNN làm bộ dự báo chính, nhưng dùng GNN để cung cấp tọa độ cấu trúc cục bộ cho token.*

3.7 Local Transformer

Trên K token của seed, RELGT dùng self-attention:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

Vì K cố định, local attention có chi phí $\mathcal{O}(K^2d)$ cho mỗi seed. Sau các lớp Transformer, mô hình dùng readout để thu được biểu diễn cục bộ:

$$h_{\text{local}}(v_i) = \sum_{j=1}^K \alpha_j h_j, \quad \alpha_j = \frac{\exp(w^\top [h_i \parallel h_j])}{\sum_k \exp(w^\top [h_i \parallel h_k])}.$$

3.8 Global centroid module

Local Transformer chỉ nhìn vào đồ thị con. Để có ngữ cảnh toàn cục, RELGT duy trì một codebook gồm B centroid. Seed representation truy vấn vào codebook:

$$h_{\text{global}}(v_i) = \text{Attn}(q(v_i), K_{\text{cb}}, V_{\text{cb}}).$$

Mỗi node được gán vào centroid gần nhất:

$$c_{v_i} = \arg \min_{b \in [B]} \|q_{v_i} - e_b\|_2.$$

Codebook có thể cập nhật bằng EMA:

$$N_b^{(t)} = \gamma N_b^{(t-1)} + (1 - \gamma) \sum_i \mathbf{1}[c_{v_i} = b], \quad (6)$$

$$e_b^{(t)} = \frac{\gamma N_b^{(t-1)} e_b^{(t-1)} + (1 - \gamma) \sum_{i:c_{v_i}=b} q_{v_i}}{N_b^{(t)}}. \quad (7)$$

3.9 Kết hợp local và global

Biểu diễn cuối cùng thường được lấy bằng:

$$h(v_i) = \text{MLP}([h_{\text{local}}(v_i) \parallel h_{\text{global}}(v_i)]), \quad \hat{y}_i = \text{Head}(h(v_i)).$$

Tùy tác vụ, loss có thể là BCEWithLogits cho phân loại nhị phân/đa nhãn hoặc MAE/MSE cho hồi quy.

3.10 Đánh giá trên RelBench

RelBench gồm nhiều cơ sở dữ liệu và 21 tác vụ, bao gồm phân loại nhị phân, hồi quy và phân loại đa nhãn. Các chỉ số thường dùng:

- ROC-AUC cho phân loại nhị phân.
- MAE cho hồi quy.
- Macro AUPRC cho phân loại đa nhãn.

Bảng 2: Kết quả thực nghiệm của RELGT trên các tác vụ RelBench tiêu biểu. Bảng được tổng hợp từ paper gốc [1]. Với MAE, giá trị thấp hơn là tốt hơn; với AUC, giá trị cao hơn là tốt hơn.

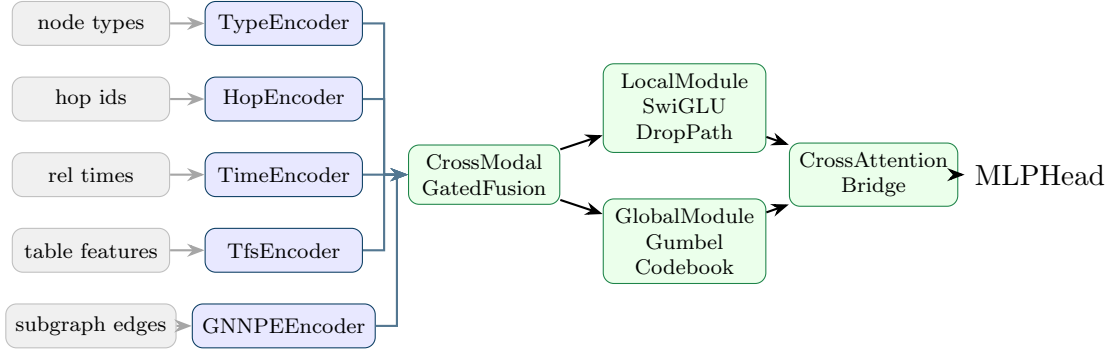
(a) MAE cho hồi quy thực thể.							(b) AUC cho phân loại thực thể.						
Dataset	Task	RDL	HGT	HGT+PE	RelGT	Gain(%)	Dataset	Task	RDL	HGT	HGT+PE	RelGT	Gain(%)
rel-fl	driver-position	4.022	4.2263	4.3921	3.9170	2.61	rel-fl	driver-dnf	0.7262	0.7077	0.7117	0.7587	4.48
rel-avito	ad-ctr	0.041	0.0462	0.0483	0.0345	15.85		driver-top3	0.7554	0.7075	0.7627	0.8352	10.56
rel-event	user-attendance	0.258	0.2635	0.2611	0.2502	2.79	rel-avito	user-clicks	0.6590	0.6376	0.6457	0.6830	3.64
rel-trial	study-adverse	44.473	45.1692	42.6484	43.9923	1.08		user-visits	0.6620	0.6432	0.6495	0.6678	0.88
	site-success	0.400	0.4428	0.4396	0.3263	18.43	rel-event	user-repeat	0.7689	0.6496	0.6536	0.7609	-1.04
rel-amazon	user-ltv	14.313	15.4120	15.8643	14.2665	0.32		user-ignore	0.8162	0.8247	0.8161	0.8157	-0.06
	item-ltv	50.053	55.8683	55.8493	48.9222	2.26	rel-trial	study-outcome	0.6860	0.5837	0.5921	0.6861	0.01
rel-stack	post-votes	0.065	0.0679	0.0680	0.0654	-0.62	rel-amazon	user-churn	0.7042	0.6643	0.6619	0.7039	-0.04
rel-hm	item-sales	0.056	0.0641	0.0639	0.0536	4.29		item-churn	0.8281	0.7797	0.7803	0.8255	-0.31
							rel-stack	user-engagement	0.9021	0.8847	0.8817	0.9053	0.35
								user-badge	0.8986	0.8608	0.8566	0.8632	-3.94
							rel-hm	user-churn	0.6988	0.6695	0.6569	0.6927	-0.87

Kết quả trong paper cho thấy RELGT có lợi thế so với các baseline GNN và tabular trong nhiều tác vụ vì nó đồng thời khai thác feature bảng, schema, thời gian, cấu trúc cục bộ và ngữ cảnh toàn cục.

4 RelGT++: Phiên bản Cải tiến

4.1 Mục tiêu thiết kế

RELGT++ giữ pipeline của RELGT nhưng thay nhiều khối bằng module mạnh hơn. Nếu RELGT trả lời câu hỏi “làm sao đưa Transformer vào dữ liệu quan hệ”, thì RELGT++ trả lời câu hỏi thực dụng hơn: “làm sao huấn luyện kiến trúc đó ổn định, giàu biểu đạt và phù hợp với code production hơn”.



Hình 2: Luồng module chính trong RELGT++.

4.2 CrossModalGatedFusion

Trong RELGT gốc, năm embedding được nối rồi chiếu tuyến tính. RELGT++ thay bằng cơ chế gate theo phương thức:

$$g_i = \sigma(W_{\text{self}}^{(i)} e_i + W_{\text{ctx}}^{(i)} \text{ctx}_i), \quad \text{ctx}_i = \frac{1}{M-1} \sum_{j \neq i} e_j.$$

Biểu diễn hợp nhất:

$$h_{\text{fused}} = \text{LN} \left(W_{\text{out}} \sum_{i=1}^M g_i \odot W_{\text{val}}^{(i)} e_i \right).$$

Ý nghĩa: mô hình không buộc feature, type, hop, time và PE có trọng số ngang nhau. Ví dụ trong tác vụ churn, time có thể quan trọng hơn; trong tác vụ dự báo sản phẩm, feature tabular và quan hệ có thể nổi bật hơn.

4.3 SwiGLU Feed-Forward Network

RELGT++ thay FFN thông thường bằng SwiGLU:

$$\text{SwiGLU}(x) = W_{\text{down}}(\text{SiLU}(W_{\text{gate}}x) \odot W_{\text{up}}x).$$

SwiGLU thường cho gradient flow tốt hơn FFN GELU đơn giản vì nhánh gate cho phép mô hình điều tiết từng chiều ẩn theo input.

4.4 Stochastic Depth

DropPath bỏ qua một số residual branch trong huấn luyện:

$$\text{DropPath}(x; p) = \begin{cases} x/(1-p), & \text{với xác suất } 1-p, \\ 0, & \text{với xác suất } p. \end{cases}$$

Trong mạng sâu, cơ chế này giảm overfitting và giúp ensemble ngầm nhiều độ sâu khác nhau.

4.5 Multi-View Readout

Thay vì chỉ dùng attention pooling, RELGT++ tổng hợp ba góc nhìn:

$$v_{\text{attn}} = \sum_j \alpha_j h_j, \quad (8)$$

$$v_{\text{mean}} = \frac{1}{K-1} \sum_j h_j, \quad (9)$$

$$v_{\text{max}} = \max_j h_j. \quad (10)$$

Sau đó:

$$h_{\text{local}} = \text{LN}(h_i + \text{MLP}([v_{\text{attn}} \parallel v_{\text{mean}} \parallel v_{\text{max}}])).$$

Attention nắm quan hệ có chọn lọc, mean giữ bức tranh trung bình, max giữ tín hiệu nổi bật hiếm.

4.6 GumbelSoftCodebook

Codebook phẳng dễ gặp *codebook collapse*: nhiều centroid không được dùng. RELGT++ thêm Gumbel noise và soft assignment:

$$l_{i,b} = -\|q_i - e_b\|_2^2, \quad s_{i,b} = \frac{\exp((l_{i,b} + g_{i,b})/\tau)}{\sum_{b'} \exp((l_{i,b'} + g_{i,b'})/\tau)}.$$

Trong đó:

$$g_{i,b} = -\log(-\log U_{i,b}), \quad U_{i,b} \sim \text{Uniform}(0, 1).$$

Nhiệt độ được anneal:

$$\tau^{(t+1)} = \max(\tau^{(t)}(1-r), \tau_{\min}).$$

Để đo mức sử dụng codebook, RELGT++ dùng entropy chuẩn hóa:

$$\mathcal{L}_{\text{entropy}} = 1 - \frac{-\sum_b p_b \log p_b}{\log B}.$$

Loss bằng 0 khi phân phối centroid đều, tiến tới 1 khi collapse.

4.7 CrossAttentionBridge

RELGT gốc nối h_{local} và h_{global} . RELGT++ dùng bridge hai chiều:

$$\alpha = \sigma \left(\frac{\langle W_q^l h_l, W_k^g h_g \rangle}{\sqrt{d}} \right), \quad \tilde{h}_l = h_l + \alpha W_v^g h_g, \quad (11)$$

$$\beta = \sigma \left(\frac{\langle W_q^g h_g, W_k^l h_l \rangle}{\sqrt{d}} \right), \quad \tilde{h}_g = h_g + \beta W_v^l h_l. \quad (12)$$

Sau đó:

$$\gamma = \sigma(W_{\text{gate}}[\tilde{h}_l \parallel \tilde{h}_g]), \quad h_{\text{bridge}} = \text{LN} \left(W_{\text{out}}(\gamma \odot \tilde{h}_l + (1-\gamma) \odot \tilde{h}_g) + h_l \right).$$

4.8 Kỹ thuật triển khai dữ liệu

Một điểm mạnh của RELGT++ là pipeline dữ liệu theo HDF5:

- lưu trước type, index, hop, relative time và edge index của từng đồ thị con;
- DataLoader đọc theo batch và scatter feature về shape $[B, K, d]$;
- encoder băng chạy theo nhóm node type để giảm số lần forward.

Nếu mã hóa từng token riêng lẻ, batch $B = 512$, $K = 300$ dẫn đến 153.600 lần encode. Group-by-type giảm xuống khoảng số loại băng, thường chỉ vài lần forward lớn.

5 Cấu hình tham khảo so sánh giữa RelGT và ReLGT++

Bảng 3: Siêu tham số tham khảo.

Tham số	Giá trị smoke/test	Giá trị đầy đủ
K token/seed	64–128	300
Embedding dim d	64–128	512
Centroid B	256–512	4096
Attention heads	4	4–8
Transformer layers	1	1–3
Learning rate	10^{-4}	10^{-4}
Weight decay	10^{-5}	10^{-5}
Dropout	0.1	0.1
Gradient clipping	1.0	1.0

6 RelGT-X: Hướng Mở rộng Nghiên cứu

6.1 Động lực

RELGT và RELGT++ đã mạnh, nhưng vẫn còn các hạn chế:

- Local attention vẫn all-pair, chưa tận dụng schema để biết cặp token nào đáng chú ý.
- Time encoder học tương quan thời gian nhưng chưa tách tín hiệu nguyên nhân và tương quan giả.
- Codebook còn phẳng, chưa phản ánh phân cấp database–table–cluster.
- Hop encoder không phân biệt các đường quan hệ khác nhau có cùng độ dài.
- Tokenization chưa thích nghi theo tác vụ.
- Biểu diễn chưa được khuyến khích bất biến qua các biến đổi schema.

RELGT-X được đề xuất như một kiến trúc kế tiếp với sáu module.

6.2 Schema-Guided Sparse Attention

Với schema $\mathcal{S} = (\mathcal{T}, \mathcal{R})$, định nghĩa ma trận reachability:

$$M_{ab}^{(r)} = \mathbf{1}[\text{tồn tại đường đi schema độ dài } \leq r \text{ từ loại } a \text{ đến loại } b].$$

Với token i, j có loại ϕ_i, ϕ_j , attention bias:

$$b_{ij} = \begin{cases} 0, & M_{\phi_i, \phi_j}^{(r)} = 1, \\ -\infty, & M_{\phi_i, \phi_j}^{(r)} = 0. \end{cases}$$

Sparse attention:

$$\text{SGSA}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + B + E_{\text{rel}} \right) V.$$

Nếu ρ là tỉ lệ cặp token hợp lệ theo schema, chi phí giảm từ $\mathcal{O}(K^2d)$ xuống $\mathcal{O}(\rho K^2d)$.

6.3 Causal Temporal Disentanglement

CTD tách biểu diễn thành thành phần nguyên nhân và tương quan giả:

$$h(v) = f_c(z_c(v)) + f_s(z_s(v)).$$

Loss gồm dự báo, bất biến qua môi trường thời gian và độc lập giữa hai phần:

$$\mathcal{L}_{\text{CTD}} = \mathcal{L}_{\text{pred}} + \lambda_1 \sum_{e \neq e'} \left\| \mathbb{E}_{v \in \mathcal{E}_e} z_c(v) - \mathbb{E}_{v \in \mathcal{E}_{e'}} z_c(v) \right\|_2^2 + \lambda_2 \left\| \text{Cov}(z_c, z_s) \right\|_F^2.$$

Mục tiêu là làm biểu diễn causal ổn định hơn khi phân phối thời gian thay đổi.

6.4 Hierarchical Codebook với Schema Priors

Thay vì codebook phẳng B centroid, HCS dùng ba tầng:

$$\text{database} \rightarrow \text{table type} \rightarrow \text{intra-table cluster} \rightarrow \text{fine centroid}.$$

Gán tầng 1 là loại bảng:

$$c_v^{(1)} = \varphi(v).$$

Tầng 2:

$$c_v^{(2)} = \arg \min_{b \in \mathcal{B}_{c_v^{(1)}}^{(2)}} \|\text{proj}^{(2)}(q_v) - e_b^{(2)}\|_2.$$

Tầng 3:

$$c_v^{(3)} = \arg \min_{b \in \mathcal{B}_{c_v^{(2)}}^{(3)}} \|\text{proj}^{(3)}(q_v) - e_b^{(3)}\|_2.$$

Schema prior có thể thêm separation loss:

$$\mathcal{L}_{\text{sep}} = \sum_{t \neq t'} \sum_{b \in \mathcal{B}_t} \sum_{b' \in \mathcal{B}_{t'}} \max(0, m - \|e_b - e_{b'}\|_2).$$

6.5 Relational Positional Encoding

Hop encoder chỉ biết độ dài đường đi, không biết đường đi đó đi qua quan hệ nào. RPE mã hóa chuỗi quan hệ:

$$\pi(v_i, v_j) = (r_1, r_2, \dots, r_k).$$

Mỗi quan hệ có embedding e_{r_l} , rồi dùng path encoder:

$$h_\pi(v_i, v_j) = \text{Transformer}_{\text{path}}(e_{r_1}, \dots, e_{r_k})_{\text{CLS}}.$$

Attention được cộng bias:

$$\text{Attn-RPE}_{ij} = \text{Attn}_{ij} + W_{\text{rpe}} h_\pi(v_i, v_j).$$

Mệnh đề 6.1. *RPE biểu đạt mạnh hơn hop encoder vì nó có thể phân biệt hai đường đi cùng độ dài nhưng khác thứ tự hoặc khác loại quan hệ, ví dụ ($\text{customer} \rightarrow \text{order} \rightarrow \text{product}$) và ($\text{product} \rightarrow \text{review} \rightarrow \text{customer}$).*

6.6 Meta-Learning Tokenization

Các tác vụ trong RelBench có nhu cầu khác nhau: task churn cần thời gian, task sales cần feature sản phẩm, task tương tác cần cấu trúc. MLT sinh bộ điều biến theo task:

$$h_{\text{token}}^{(\tau)} = O(g_\tau \odot [h_{\text{feat}} \parallel h_{\text{type}} \parallel h_{\text{hop}} \parallel h_{\text{time}} \parallel h_{\text{pe}}]),$$

$$g_\tau = \sigma(\text{MLP}_{\text{task}}(c_\tau)),$$

trong đó c_τ mô tả loại tác vụ, số lớp, phân phối nhãn hoặc thống kê schema liên quan.

7 Thảo luận

7.1 Ý nghĩa khoa học

RELGT cho thấy Graph Transformer có thể dùng hiệu quả trên dữ liệu quan hệ nếu bài toán tokenization được thiết kế đúng. Đóng góp không nằm ở việc áp dụng Transformer một cách trực tiếp, mà ở cách biến node trong cơ sở dữ liệu quan hệ thành token có đủ năm lớp thông tin: nội dung, kiểu, khoảng cách, thời gian và vị trí.

RELGT++ nhấn mạnh một điểm khác: với kiến trúc phức tạp, hiệu quả thực tế phụ thuộc nhiều vào các chi tiết kỹ thuật như fusion, readout, codebook update, loss phụ và pipeline dữ liệu. Một ý tưởng tốt có thể yếu nếu triển khai không ổn định; ngược lại, một loạt cải tiến nhỏ nhưng đúng chỗ có thể làm mô hình dễ train hơn đáng kể.

RELGT-X mở rộng từ “học trên graph quan hệ” sang “học có prior về schema và thời gian”. Đây là bước tự nhiên vì cơ sở dữ liệu quan hệ không chỉ là graph, mà là graph được sinh ra bởi schema có ý nghĩa.

7.2 Giới hạn

Các hướng mở rộng trong RELGT-X cần được kiểm chứng cẩn thận. SGSA phụ thuộc vào chất lượng schema mask; CTD cần định nghĩa môi trường thời gian hợp lý; HCS cần tránh phân mảnh codebook; CSA cần augmentation không phá ngữ nghĩa. Do đó, RELGT-X nên được phát triển theo từng module và luôn có ablation.

7.3 Kết luận

RELGT là một bước tiến quan trọng cho Relational Deep Learning vì nó đưa Transformer vào dữ liệu quan hệ bằng một thiết kế token hóa phù hợp. RELGT++ củng cố kiến trúc đó bằng các cải tiến thực dụng về fusion, readout, codebook và huấn luyện. RELGT-X là định hướng nghiên cứu tiếp theo, khai thác sâu hơn schema, nhân quả thời gian và khả năng transfer giữa tác vụ.

Thông điệp chính của toàn bộ chuỗi này là: để học tốt trên cơ sở dữ liệu quan hệ, mô hình cần hiểu đồng thời **bảng nào, quan hệ nào, khi nào, ở đâu trong đồ thị con**, và **ngữ cảnh toàn cục nào** đang ảnh hưởng đến dự báo.

Tài liệu

- [1] Dwivedi, V. P., Jaladi, S., Shen, Y., López, F., Kanatsoulis, C. I., Puri, R., Fey, M., & Leskovec, J. (2025). *Relational Graph Transformer*. arXiv:2505.10960.
- [2] Robinson, J. et al. (2023). *RelBench: A Benchmark for Deep Learning on Relational Databases*. NeurIPS.
- [3] Vaswani, A. et al. (2017). *Attention Is All You Need*. NeurIPS.
- [4] Hamilton, W., Ying, Z., & Leskovec, J. (2017). *Inductive Representation Learning on Large Graphs*. NeurIPS.
- [5] Hu, Z. et al. (2020). *Heterogeneous Graph Transformer*. WWW.
- [6] Rampásek, L. et al. (2022). *Recipe for a General, Powerful, Scalable Graph Transformer*. NeurIPS.
- [7] Xu, K. et al. (2019). *How Powerful are Graph Neural Networks?* ICLR.
- [8] Jang, E., Gu, S., & Poole, B. (2017). *Categorical Reparameterization with Gumbel-Softmax*. ICLR.
- [9] Finn, C., Abbeel, P., & Levine, S. (2017). *Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks*. ICML.
- [10] Chen, T. et al. (2020). *A Simple Framework for Contrastive Learning of Visual Representations*. ICML.
- [11] Schölkopf, B. et al. (2021). *Toward Causal Representation Learning*. Proceedings of the IEEE.
- [12] Kipf, T. N., & Welling, M. (2017). *Semi-Supervised Classification with Graph Convolutional Networks*. ICLR.
- [13] Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., & Dahl, G. E. (2017). *Neural Message Passing for Quantum Chemistry*. ICML.
- [14] Velickovic, P. et al. (2018). *Graph Attention Networks*. ICLR.
- [15] Schlichtkrull, M. et al. (2018). *Modeling Relational Data with Graph Convolutional Networks*. ESWC.

- [16] Ying, C. et al. (2021). *Do Transformers Really Perform Bad for Graph Representation?* NeurIPS.
- [17] Corso, G., Cavalleri, L., Beaini, D., Lio, P., & Velickovic, P. (2020). *Principal Neighbourhood Aggregation for Graph Nets*. NeurIPS.
- [18] Huang, Y. et al. (2024). *On the Stability of Expressive Positional Encodings for Graphs*. ICLR.
- [19] Deng, C., Yue, Z., & Zhang, Z. (2024). *Polynormer: Polynomial-Expressive Graph Transformer in Linear Time*. ICLR.
- [20] Fu, D. et al. (2024). *VCR-Graphormer: A Mini-batch Graph Transformer via Virtual Connections*. ICLR.
- [21] Bar-Shalom, G., Bevilacqua, B., & Maron, H. (2024). *Subgraphormer: Unifying Subgraph GNNs and Graph Transformers via Graph Products*. ICML.
- [22] Canturk, S. et al. (2024). *Graph Positional and Structural Encoder*. ICML.
- [23] Black, M., Wan, Z., Mishne, G., Nayyeri, A., & Wang, Y. (2024). *Comparing Graph Transformers via Positional Encodings*. ICML.
- [24] Kanatsoulis, C., Choi, E., Jegelka, S., Leskovec, J., & Ribeiro, A. (2025). *Learning Efficient Positional Encodings with Graph Neural Networks*. ICLR.
- [25] Bao, L., Jin, E., Bronstein, M., Ceylan, I. I., & Lanzinger, M. (2025). *Homomorphism Counts as Structural Encodings for Graph Learning*. ICLR.
- [26] Chen, T., Kanatsoulis, C., & Leskovec, J. (2025). *RelGNN: Composite Message Passing for Relational Deep Learning*. ICML.
- [27] Khalid, I., & Schockaert, S. (2025). *Systematic Relational Reasoning With Epistemic Graph Neural Networks*. ICLR.